Gestiunea proceselor

Ce este un proces

Un proces este un program sau o parte a unui program, în execuție

Fiecare proces are propriul spațiu de memorie, și resurse de sistem

Procesele sunt unitatea fundamentală de execuție în Unix

Un sistem modern Unix/Linux poate avea sute sau mii de procese care rulează simultan.

Ciclul de viață al unui proces Unix

Creare: Procesul este generat (fork/exec)

Pregătit de execuție: Procesul este gata de rulare dar așteaptă timp CPU

În execuție: Procesul execută activ pe CPU

Blocat (în așteptare): Procesul așteaptă un eveniment (I/O, semnal, etc.)

Terminat: Procesul și-a încheiat execuția

Crearea proceselor

Crearea se face folosind apelul de sistem **fork()**

- Creează un proces copil ca o copie a părintelui
- Procesul copil primește un nou PID
- Copilul moștenește descriptorii de fișiere deschiși și alte proprietăți

De obicei e urmat de **exec()** pentru a înlocui imaginea procesului.

fork() + **exec()** = metoda standard Unix de creare a noilor procese

Ierarhia proceselor

Arborele de Procese

Fiecare proces (cu excepția init/PID 1) are un părinte

Procesele formează o structură ierarhică arborescentă

Exemplu:

```
init(1)─┬─sshd(463)───sshd(26594)───bash(26601)───mc(27949)
        ├─cron(586)
        ├─getty(587)
        └─getty(588)
```

Procesul *Init* (frecvent *systemd* în sistemele Linux moderne) este strămoșul tuturor proceselor utilizator.

Identificatori ai proceselor

PID: ID-ul procesului - identificator unic pentru fiecare proces

PPID: ID-ul procesului părinte - PID-ul părintelui procesului

PGID: ID de Grup de procese - pentru distribuirea semnalelor și controlul joburilor

SID: ID de Sesiune - grupează mai multe grupuri de procese

Identificatori ai proceselor

Procesele copil sunt create prin fork()

Procesele copil moștenesc multe atribute de la părinți:
- Variabile de mediu
- Descriptori de fișiere deschiși
- Handleri de semnale
- Limite de resurse

Părintele poate aștepta terminarea copilului

Procesele copil rulează independent de părinți și au propriile spații de memorie

Apelurile sistem wait(), waitpid() permit proceselor-părinte să se sincronizeze cu terminarea copiilor lor și să colecteze starea lor de ieșire.

Terminarea proceselor

Terminare normală: Procesul apelează exit() cu cod de stare
Terminare anormală: Procesul primește un semnal (ex. SIGTERM, SIGKILL)

Statusul de ieșire (0-255) indică succes/eșec (0 = succes)

Părintele poate recupera statusul de ieșire prin familia de apeluri wait()

Resursele sunt eliberate, dar informații despre proces (intrarea sa în tabela de procese) rămân până când este apelat wait()

Semnale precum SIGTERM permit o închidere elegantă în timp ce SIGKILL forțează terminarea imediată.

Procese orfane

Un **proces orfan** este un proces al cărui părinte s-a terminat

Procesele orfane sunt de obicei “adoptate” de un proces special
- De obicei acesta este init (PID 1), care devine noul părinte (PPID = 1)

Procesele orfane continuă să ruleze normal
(NU se termină la terminarea părintelui !)

Procese orfane – mecanisme alternative

În unele implementări Unix:
procesele orfane **nu** sunt întotdeauna preluate de procesul init (PID 1)

Un proces se poate înregistra ca *subreaper* și poate adopta descendenții săi orfani

- Mecanism introdus în Linux 3.4 prin prctl(PR_SET_CHILD_SUBREAPER)

(prctl e o funcție sistem cu care se pot configura diverse aspecte ale gestiunii proceselor și firelor de execuție)

- FreeBSD oferă funcționalitate similară cu procctl(PROC_REAP_ACQUIRE)

Scop:

- Permite supravegherea proceselor în cadrul unor subsisteme izolate

  Exemplu: un proces supervizor se inregistrează ca subreaper pentru a menține controlul asupra proceselor sale worker

- Utilizat frecvent în medii container (ex: Docker)

  managerul containerului se inregistreaza ca subreaper deoarece trebuie să urmărească și să gestioneze toate procesele dintr-un container

Procese zombie

Un proces **zombie** (defunct) e un proces care s-a terminat dar pentru care părintele nu a apelat wait()

Intrarea din tabela de procese rămâne ocupată (dar se eliberează memoria)

Apare cu “Z” în status-ul procesului, afișat de comanda ps

Procesul rămâne vizibil ca “zombie” până când:

- Părintele apelează wait(), sau
- Părintele se termină (zombie-ul este adoptat de init sau subreaper, care va apela wait())

Prea multe procese zombie pot umple tabela de procese

Grup de procese

Un grup de procese este o colecție de procese, care vor avea același PGID

Create prin apelul sistem setpgid()

Utilizate pentru controlul joburilor și distribuirea semnalelor

Semnalele trimise către PGID sunt livrate tuturor membrilor

Terminalul
- are un grup de procese în prim-plan (foreground)
- pot exista mai multe grupuri de procese în fundal (background), care **nu** au acces la intrarea de la tastatură

Procese “daemon”

Procesele “daemon” sunt procese care rulează în fundal (background) pe termen lung, fără terminal de control

Convențional au nume care se termină în “d” (httpd, sshd, systemd)

Pașii uzuali de creare a unui proces “daemon”:

- Se apelează fork(), iar părintele se termină imediat cu exit() (procesul copil devine orfan)
- Procesul copil apelează setsid() pentru a deveni lider de sesiune
- Închide descriptorii standard de fișiere (stdin, stdout, stderr)
- Schimbă directorul de lucru la /
- Setează un *umask* restrictiv (*umask*=o mască de biți specifică procesului, pe care UNIX o folosește pentru a determina permisiunile fișierelor nou create de acel proces)
- Adesea execută fork din nou (“double-fork”) pentru a asigura că nu I se poate asocia niciun terminal

Procesele “daemon” pot crea un fișier PID în /var/run/ pentru a preveni instanțe multiple

Izolarea proceselor

În sens restrâns:

- sistemul de operare asigură faptul că procesele rulează independent, au spații de memorie distincte etc.

În sens larg:

- crearea de către sistemul de operare a unor spații separate, în fiecare dintre ele putând exista mai multe procese, care rulează ca și cum ar fi în interiorul unor mașini distincte
- este necesar pentru implementarea sistemelor care creează containerizare (ex.: Docker)

Containerizare – namespaces

Mecanismul principal: **spații de nume (namespaces)**
→ partiționează resursele kernel-ului, astfel încât un set de procese vede un set de resurse, în timp ce alt set de procese vede un set diferit de resurse

Tipuri de spații de nume folosite în containerizare:

- **Spațiu de nume PID (ID Proces)**: Procesele dintr-un container au propriul arbore de procese, începând cu PID 1. Ele nu pot vedea procesele din afara spațiului lor de nume PID (din alte containere sau de pe sistemul gazdă).
- **Spațiu de nume rețea (Net)**: Fiecare container primește propria stivă de rețea: propria adresă/adrese IP, tabele de rutare, numere de port etc.
- **Spațiu de Nume de montare (Mnt)**: Oferă o ierarhie locală, izolată, a sistemului de fișiere. Un container are propriul său sistem de fișiere rădăcină.
- **Spațiu de Nume UTS (Unix Timesharing System)**: Izolează numele de gazdă (hostname) și numele de domeniu.
- **Spațiu de Nume IPC (Inter-Process Communication)**: Izolează resursele IPC System V (cum ar fi memorie partajată, semafoare) și cozile de mesaje POSIX
- **Spațiu de Nume Utilizator (User)**: Mapează ID-urile de utilizator și de grup din interiorul containerului la ID-uri de utilizator și de grup diferite, adesea neprivilegiate, pe sistemul gazdă.
- **Spațiu de Nume Cgroup (Control Groups)**: Izolează domeniul grupurilor de control, permițând proceselor din interiorul unui container să vadă o ierarhie cgroup locală, relevantă doar pentru ele.

Containerizare – alte mecanisme

Pe lângă spațiile de nume (namespaces), există și alte mecanisme folosite în containerizare.

Exemple:

Grupuri de Control (cgroups): oferă limitarea și contorizarea resurselor. Ele permit sistemului gazdă să aloce și să restricționeze cantitatea de timp CPU, memorie, I/O pe blocuri, lățime de bandă de rețea etc., pe care un container (ca grup de procese) o poate consuma. Acest lucru împiedică un singur container să monopolizeze resursele gazdei și să afecteze alte containere sau gazda însăși. Este o parte critică a asigurării distribuției echitabile a resurselor și prevenirii scenariilor de tip "denial-of-service" între containere.

Seccomp (Secure Computing Mode): restricționează apelurile sistem (system calls) pe care un proces le poate face. Runtime-urile de containere (cum ar fi Docker) aplică profile seccomp implicite care dezactivează multe apeluri de sistem potențial periculoase sau inutile pentru procesele containerizate, reducând suprafața de atac a kernelului expusă containerului.